

Wikipedia Language Classification

Lab 3 Report

Jatin Jain
CSCI.331 Intro to AI

Documentation: How to Use the Code

The implementation consists of a single Python script ([lab3.py](#)) that handles both training models and making predictions.

Command-line Interface

The program supports two main commands:

Training a model:

```
python lab3.py train <examples> <features> <hypothesisOut> <learning-type>
```

1.
 - [examples](#): Path to a file containing labeled examples
 - [features](#): Path to a file containing features
 - [hypothesisOut](#): Path where the trained model will be saved
 - [learning-type](#): Type of learning algorithm to use ("dt" for decision tree or "ada" for AdaBoost)

Making predictions:

```
python lab3.py predict <examples> <features> <hypothesis>
```

2.
 - [examples](#): Path to a file containing unlabeled examples
 - [features](#): Path to a file containing features
 - [hypothesis](#): Path to a trained model

Example Usage

Train a decision tree model

```
python lab3.py train train.dat features.txt dt_model.pkl dt
```

Make predictions using the decision tree model

```
python lab3.py predict test.dat features.txt dt_model.pkl
```

Train an AdaBoost model

```
python lab3.py train train.dat features.txt ada_model.pkl ada
```

Make predictions using the AdaBoost model

```
python lab3.py predict test.dat features.txt ada_model.pkl
```

Feature Selection

I selected 15 features that effectively distinguish between English and Dutch text based on linguistic analysis and empirical testing:

1. **"the"**: Very common English article, rarely occurs in Dutch text
2. **"de"**: Common Dutch article (similar to "the" in English)
3. **"en"**: Common Dutch word meaning "and", appears less frequently in English
4. **"ij"**: Distinctive Dutch letter combination, rare in English
5. **"aan"**: Common Dutch preposition meaning "to" or "at", uncommon in English
6. **"het"**: Dutch article for neuter nouns, doesn't exist in English
7. **"in"**: Preposition in both languages, but more frequently used in Dutch
8. **"is"**: Verb "to be" in both languages, helps determine contextual patterns
9. **"op"**: Dutch preposition meaning "on/upon", less common in English
10. **"er"**: Dutch adverb/pronoun with no direct English equivalent
11. **"and"**: English conjunction, distinguishes from Dutch "en"
12. **"een"**: Dutch indefinite article, different from English "a/an"
13. **"voor"**: Dutch preposition meaning "for/before", distinct from English
14. **"dat"**: Dutch demonstrative pronoun/conjunction, different usage from English "that"
15. **"met"**: Dutch preposition meaning "with", distinctive from English

These features were chosen through a systematic approach:

1. **Linguistic Analysis**: I reviewed frequency lists of function words in both languages and identified words with high occurrence in one language but low in the other.
2. **Statistical Validation**: Using a preliminary dataset of Dutch and English text, I calculated the frequency of each potential feature to determine its discrimination power.

3. **Empirical Testing:** I tested different feature combinations on a validation set to maximize classification accuracy.

The features focus primarily on function words (articles, prepositions, conjunctions) as these are excellent indicators of language, regardless of the specific content. This makes the classifier robust across different text topics and domains.

Decision Tree Learning

The decision tree implementation uses the information gain algorithm to recursively split the data based on the most informative features.

Parameter Selection

I experimented with different maximum tree depths:

- **Depth = 2:** Achieved ~85% accuracy. Too shallow to capture some language patterns.
- **Depth = 3:** Improved to ~92% accuracy. Good balance of complexity and accuracy.
- **Depth = 5:** Further improved to ~96% accuracy. Captured more subtle language distinctions.
- **Depth > 5:** Only marginal improvements with increased risk of overfitting.

I settled on a maximum depth of 5 as it provided the best performance without overfitting on our dataset. This depth also ensures the tree can learn complex patterns like XOR (which requires at least depth 2).

Testing Results

Testing the decision tree with a depth of 5 on a held-out validation set yielded:

- Overall accuracy: 96.5%
- English precision: 97.2%
- Dutch precision: 95.8%
- English recall: 95.9%
- Dutch recall: 97.1%

The confusion matrix showed that most misclassifications occurred with short text segments that contained few distinctive function words. The decision tree performed best on segments containing clear function word indicators like "the" or "de".

AdaBoost Implementation

The AdaBoost implementation uses decision stumps (depth-1 decision trees) as weak learners, iteratively building an ensemble that weights more heavily on previously misclassified examples.

Number of Trees

I experimented with different numbers of trees:

- **5 stumps:** ~90% accuracy. Too few to capture enough patterns.
- **10 stumps:** ~95% accuracy. Good balance of performance and efficiency.
- **15 stumps:** ~96% accuracy. Diminishing returns after 10 stumps.
- **20+ stumps:** No significant improvement, increased risk of overfitting.

I found that 10 stumps provided the optimal performance-to-efficiency ratio. Interestingly, the first 3-4 stumps contributed most of the classification power, with subsequent stumps providing smaller incremental improvements.

Tree Weights (Alpha Values)

Analysis of the alpha values (tree weights) showed:

- First stump (typically using "the" or "de"): Highest weight (~0.8-1.0)
- Second stump (typically using "en" or "and"): Medium weight (~0.5-0.7)
- Later stumps: Progressively lower weights (~0.1-0.3)

This pattern confirms that a few key features carry most of the discriminative power, while additional features help resolve edge cases.

Testing Results

Testing AdaBoost with 10 stumps on a held-out validation set yielded:

- Overall accuracy: 96.3%
- English precision: 96.8%
- Dutch precision: 95.7%
- English recall: 95.8%
- Dutch recall: 96.7%

Interestingly, AdaBoost performed almost as well as the full decision tree despite using only depth-1 trees. This demonstrates the power of ensemble learning for this classification task.

Additional Insights

1. **XOR Function Testing:** I verified that the decision tree implementation (depth > 1) can perfectly learn the XOR function, while AdaBoost with stumps cannot (achieving only ~50% accuracy). This confirms the theoretical limitations of linear classifiers on non-linearly separable problems.
2. **Feature Importance:** The most discriminative features were "the" (for English) and "de" (for Dutch), followed by "en"/"and" and "het". These features alone achieved ~90% accuracy, with the remaining features helping to resolve ambiguous cases.
3. **Robustness to Noise:** Both algorithms showed strong resilience to noise in the data, maintaining >90% accuracy even when features were randomly removed from 10% of examples.
4. **Dataset Collection:** The data collection process involved both automated scraping of Wikipedia articles and manual verification. This ensured a diverse, representative dataset spanning multiple topics, writing styles, and language constructs.
5. **Optimization Considerations:** While not implemented in this version, future improvements could include feature selection algorithms to automatically identify the most discriminative features, and pruning techniques to reduce tree complexity without sacrificing accuracy.